

Module 4 — Subagents

Lesson 4.2

Writing a subagent definition

A subagent is a markdown file with frontmatter. Drop it in `.claude/agents/`, pick its tools, write its system prompt. That's it.

Why it matters

Built-in agent types (general-purpose, Explore, Plan) cover the basics. Custom subagents earn their keep when you have a repeated specialized task — code review with your team's rubric, PR triage with your team's labels, database schema review against your team's conventions. Define once; use forever.

Where they live

Path	Scope
<code><project>/.claude/agents/*.md</code>	Project-scoped (committed; shared with team)
<code>~/.claude/agents/*.md</code>	User-global

Project-scoped wins on name collisions. Filename → agent name. `pr-reviewer.md` becomes `pr-reviewer`.

The schema

```
---
name: pr-reviewer
description: |
  Independent reviewer for pull requests. Use when you want a fresh
  perspective on a diff – independent of your reasoning so far.
  Specializes in security, correctness, and team conventions.
tools: Read, Glob, Grep, Bash(git diff:*), Bash(gh pr view:*)
model: claude-opus-4-7
---
```

You are a focused code reviewer for the <Project> team.

What you do

1. Read the diff under review (you'll receive it as input or fetch via `gh`).
2. Read related files for context (use Read and Grep).
3. Evaluate against the rubric below.
4. Return a structured report.

Rubric

- Correctness: does the change do what its description claims?
- Safety: any data-loss, race, or security risks?
- Conventions: matches our style? (snake_case DB, camelCase API, AppError subclasses for errors)
- Test coverage: is there a test for the new behavior? For the bug fix?
- Scope: any unrelated changes that should be pulled out?

Output format

- Verdict: APPROVE / REQUEST CHANGES / BLOCK
- Top issues (max 5), each as: <file>:<line> – <one-sentence issue>
- One-sentence overall assessment.

Style

- Lead with verdict.
- No preamble. No "great work."
- Cite file:line for every issue.

Frontmatter fields

Field	Purpose
<code>name</code>	The slug used to invoke (matches filename)
<code>description</code>	Shown when listing agents; also used by the dispatcher to pick this agent automatically — write a clear, specific description
<code>tools</code>	Comma-separated allowed tools; same syntax as <code>permissions.allow</code>
<code>model</code>	Optional model override

Body

The body is the agent's system prompt. Treat it like an instruction manual for a focused contractor.

Design principles for good subagents

1. Narrow scope

One agent, one job. A `code-reviewer` that also writes patches is two agents in a trenchcoat. Split them.

2. Constrain tools

Read-only agents (no Edit, no Write, no destructive Bash) are safer and easier to reason about. If your agent doesn't need to edit, don't give it Edit.

3. Specify the output shape

Tell the agent the exact format you want back. "Return X, then Y, then Z" — not "give me a thorough analysis."

4. Cap verbosity

Almost every subagent should have a word limit in either its definition or its invocation brief. "Under 300 words" is rarely too tight.

5. Write the description for the dispatcher

The `description` field isn't just for humans. Claude's main loop reads it when deciding whether to auto-spawn this agent based on the user's request. Be specific:

- Bad: "Helps with reviews."
- Good: "Use when the user wants an independent second opinion on a diff. Specializes in correctness, security, and team conventions. Returns a structured verdict."

Common patterns

A focused researcher

```
---
name: incident-timeline-builder
description: Builds a timeline of events from logs and metrics around an incident
tools: Read, Glob, Grep, Bash(date:*), Bash(jq:*)
---
```

Build an incident timeline. Inputs: an incident window (start/end ISO) and log paths.

Output:

- Timeline: bulleted list, `HH:MM:SS – event` format, ordered.
- Inflection points (errors, retries, recoveries) flagged with `███`.
- One-paragraph narrative at the end.

Hard rules:

- Cite every event with its log file:line.
- Don't speculate; quote the log line.

A code-aware tester

```
---
name: test-coverage-auditor
description: Finds untested code paths in changed files
tools: Read, Glob, Grep
---
```

For each file in the supplied diff:

1. Identify all exported functions/methods.
2. Find their tests (by searching for the symbol in `\*.test.\*` or `tests/`).
3. Report symbols with no test, ranked by complexity (lines + branches).

Output: a markdown table – symbol, file:line, complexity, has_test.

A doc-aware refactorer

```
---
name: jsdoc-updater
description: Updates JSDoc comments to match current function signatures
tools: Read, Edit, Grep, Glob
model: claude-haiku-4-5
---
```

For the supplied file(s):

1. For each function with JSDoc, compare the documented params/returns to the actual signature.
2. Update mismatches.
3. Do not add new JSDoc to undocumented functions.
4. Do not change function bodies.

Output: file paths edited and count of comments updated.

Note the Haiku model choice — for narrow mechanical work, smaller models are faster and cheaper.

Try it

Runnable example: `examples/03-pr-reviewer-subagent`.

- 1 Copy the example into a project.
- 2 Tune the rubric to your team's conventions.
- 3 Invoke it on an existing PR.
- 4 Compare its output to `/review`. Note where the custom agent wins (your conventions).

Gotchas

- **A vague description means the dispatcher won't pick it.** Be specific about *when* the agent applies.
- **``tools`` is enforced.** If you forget to grant a tool the agent needs, it'll fail mid-task.
- **The body is loaded as the agent's system prompt** — every word costs context for the agent. Don't pad.
- **Agents can't call other agents** in most setups. Don't design a hierarchy.

Further reading

- 4.3 Prompting subagents — invocation brief design
- `examples/03-pr-reviewer-subagent` — runnable
- Official subagents docs: <https://docs.claude.com/en/docs/claude-code/sub-agents>

Next

→ 4.3 Prompting subagents effectively